

BC Deployment Analyzer

A production-ready multi-agent documentation pipeline for
Business Central Per-Tenant Extensions on Microsoft Foundry

Solution design · Agent architecture · Observability · Evaluation · Workflow

Build and Scale AI Agents with Microsoft Foundry — Level 3: Architect
Final assignment · Hannes K, Information & Business Solutions Svenska AB (InBiz)
2026-09-17 · Repository: github.com/hannisky/BC-Deployment-Analyser

Contents

Contents	2
Executive summary	3
Architecture at a glance	4
1. Solution design	5
1.1 Business problem	5
1.2 Intended users.....	5
1.3 The agents	5
1.4 Tools, data sources and knowledge	6
1.5 Why a multi-agent approach.....	7
1.6 Information flow between the agents.....	7
2. Production-readiness plan.....	9
2.1 Observability strategy.....	9
2.2 Evaluation strategy	10
2.3 Governance and reliability	11
3. End-to-end workflow.....	13
3.1 Sequence of agent interactions.....	13
3.2 Information passed between steps	14
3.3 Discovery and version validation.....	14
3.4 Final output delivered to the business	14
3.5 Deployment, monitoring, evaluation and improvement over time	15
4. From prototype to production — reflection	17
Appendix A — Repository map.....	17

Executive summary

A Microsoft Dynamics 365 Business Central partner maintains dozens of customer-specific extensions (Per-Tenant Extensions, PTEs) in GitHub. The knowledge of what those extensions do, which version is live in each customer environment, and what changed last sprint lives in AL source code and in developers' heads. Hand-written implementation documents go stale the day after they are written.

The BC Deployment Analyzer keeps that documentation alive automatically. Deterministic Python discovery inventories the customer's repositories and Business Central environments and validates published versions against source control. Two specialised Microsoft Foundry agents then collaborate: **bc-change-tracker** reads the pull requests merged since the last run and rates the documentation impact; **bc-code-analyst** reads the AL source through a deterministic analysis tool and writes — or surgically updates — the Swedish "Anpassningar" documentation for each app. The Markdown lands in GitHub, the Microsoft 365 Copilot GitHub connector indexes it, and consultants ask Copilot instead of reading code.

The solution productionises an existing proof of concept (one plain OpenAI call per repository, no tracing, no evaluation) using the lifecycle taught in the course: setup → build agents → monitor → evaluate → orchestrate. Every phase is implemented as runnable code in the repository; this document describes the design decisions behind it.

Course requirement	Where it is addressed
Business problem, users, ≥2 agents, tools/data, why multi-agent	Section 1 — Solution design
Observability strategy	Section 2.1
Evaluation strategy	Section 2.2
Governance and reliability	Section 2.3
End-to-end workflow: sequence, information passed, tool calls, output, operations	Section 3
Reflection: prototype → production	Section 4

Architecture at a glance

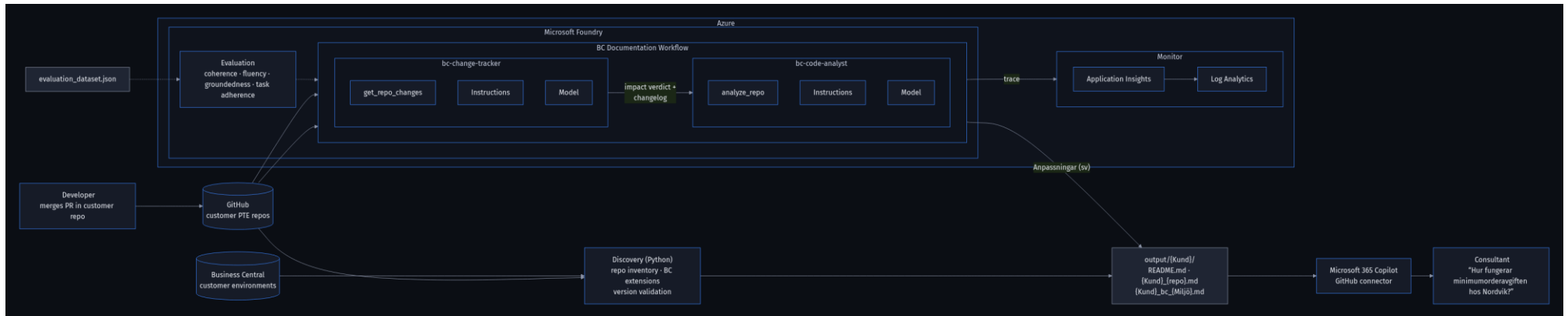


Figure 1 — End-to-end architecture: GitHub and Business Central feed deterministic discovery and the two Foundry agents; output is indexed by Microsoft 365 Copilot; traces flow to Application Insights; evaluation runs against a versioned dataset.

Reading the figure left to right: a developer merges a pull request in a customer repository. Discovery (plain Python) refreshes the repository inventory and queries Business Central. Inside Microsoft Foundry the change tracker hands an impact verdict and changelog to the code analyst, which produces the Swedish documentation section. Every agent run is traced to Application Insights and Log Analytics; the evaluation dataset scores the agents before each release. The generated Markdown is committed to GitHub and surfaced to consultants through Microsoft 365 Copilot.

1. Solution design

1.1 Business problem

The partner develops and operates customer-specific extensions for Business Central. Each customer has three to ten GitHub repositories of AL code and one or more Business Central environments (production, sandbox). Four recurring problems motivated the solution:

- **Knowledge is locked in code.** Only the developer who wrote a codeunit knows what it does. Support consultants either read AL or ask around.
- **Documentation decays.** The partner's hand-written **Implementationsdokumentation** (installed extensions, process descriptions, customisations) is accurate on the day it is written. A real reference document was missing an entire feature (*CustomCreditLimitMgt*) that existed in the repository.
- **Nobody knows what is live.** Whether the version in *main* is the version installed in the customer's production environment is checked manually, if at all — a source of surprises during upgrades.
- **Change history is invisible to non-developers.** Merged pull requests explain **why** something changed, but consultants never see them.

The measurable goal: every customer app has current Swedish documentation within one day of a merged pull request, every published version is validated against source control, and consultants can answer "how does X work at customer Y" from Copilot without reading code.

1.2 Intended users

User	Needs	Touchpoint
Support / application consultants	"How does the minimum order fee work at Nordvik?" · "Which apps does customer X have and are they up to date?"	Microsoft 365 Copilot chat (GitHub connector indexes the generated Markdown), or the Markdown directly in GitHub
Project managers / customer leads	Release notes per customer per sprint in plain Swedish	<i>*Ändringshistorik*</i> section per app; <i>*Senaste ändringar*</i> on the customer index page
Developers	Zero extra work; a safety net that flags unpublished versions and undocumented objects	Nothing to do — merging a PR triggers the update
Operations (partner IT)	Cost, failures and quality of the pipeline	Application Insights dashboards, KQL queries, evaluation reports

1.3 The agents

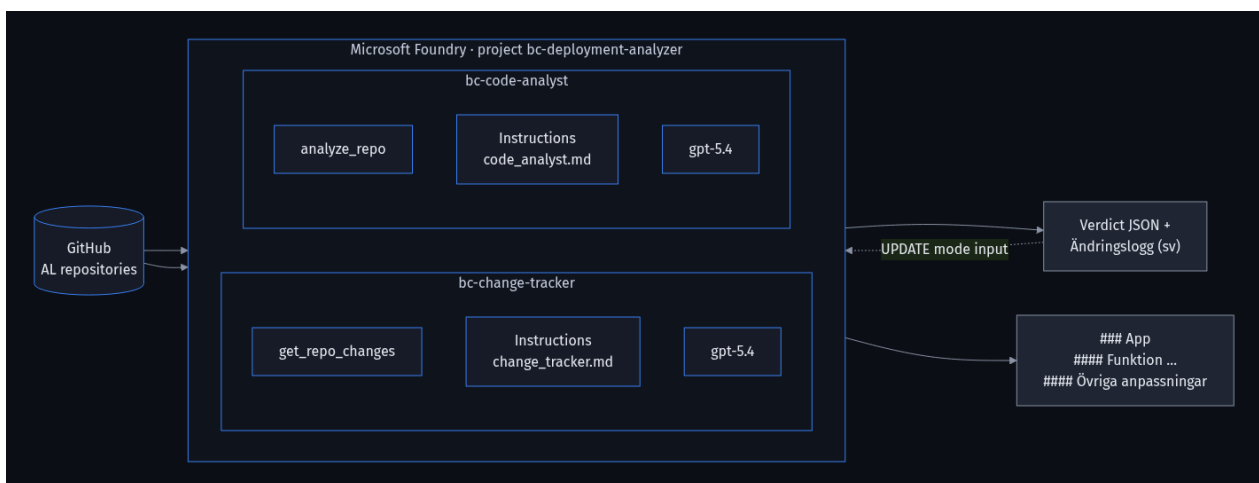


Figure 2 — The two agents, their tools and instruction files inside the Foundry project. The tracker's verdict feeds the analyst's update mode.

	bc-change-tracker	bc-code-analyst
Responsibility	Explain what changed in a repository between two commits and rate how much of the documentation is affected	Describe what an AL app does for consultants, in Swedish, following the partner's documentation standard
Question answered	What changed since we last documented this, and does it matter?	What does this app do, today?
Tool (FunctionTool)	<code>get_repo_changes(org, repo, base_sha, head_sha)</code>	<code>analyze_repo(org, repo, branch)</code>
Input size	Small — a diff, PR titles/bodies, patches	Large — a whole PTE repository
Output contract	JSON verdict (<code>impact: none/minor/major, affected_features, new_objects, requires_full_regeneration, summary_sv</code>) + Swedish changelog entry	### App → ### Feature (AL object) → ### Övriga anpassningar Markdown section; full or update mode
Instructions	<code>agents/instructions/change_tracker.md</code>	<code>agents/instructions/code_analyst.md</code>
Model	gpt-5.4 (GlobalStandard deployment in the Foundry project)	gpt-5.4

Both agents are `PromptAgentDefinition` agents created with `create_version()`. They are versioned, visible in the Foundry portal under Build → Agents, and reused by name from the orchestrator — created once, never re-created per run.

1.4 Tools, data sources and knowledge

The design principle is **tools are code, agents are prose**. Everything decidable by rules is decided in Python, and the agent only reasons over the structured result. This is the same `check_thresholds` pattern used in the course labs, applied to source code analysis.

Component	Type	Used by	Grounding role
<code>common/al_analysis.py</code>	Deterministic parser	<code>analyze_repo, get_repo_changes</code>	Object types/ids/names, table fields, page actions, API properties, procedures, <code>[EventSubscriber]</code> attributes; classifies codeunits into three archetypes (logic → own section, subscribers → "Övriga anpassningar", install/upgrade → skipped); detects commented-out bodies
GitHub REST API	Data source (read-only PAT)	both tools, discovery	Source of truth for code, tree, compare, merged pull requests
Business Central Automation API	Data source (client credentials via GDAP)	<code>discover_bc_extensions.py</code>	Installed extensions and versions per environment
<code>discovery/repos_config.json</code>	Inventory	discovery, orchestrator	Which repos belong to which customer; app id/name/version; head SHA
<code>output/{Kund}/.doc_state.json</code>	State	orchestrator	Last documented SHA and version per repo → incremental runs
<code>known_extensions.json</code>	Curated lookup (optional)	BC report	Descriptions and vendor links for third-party ISV apps
<code>agents/instructions/*.md</code>	Governance	agents	Format standard derived from the partner's reference document; versioned in git

There is deliberately **no vector store or retrieval-augmented generation in version 1**. The ground truth is always the current code, which fits in the model's context per repository. A knowledge base of past documents would risk describing removed features. A curated knowledge base for process descriptions (Section 2 of the partner's template, `*Flödesbeskrivning*`) is a planned extension, see Section 3.4.

1.5 Why a multi-agent approach

1. **Different questions on different data.** The tracker reasons about a small diff and must be precise about deltas; the analyst reasons about a whole repository and must be complete. One prompt asked to be both surgical and exhaustive degrades at both.
2. **Cost and latency.** The orchestrator skips unchanged repositories with a SHA comparison (zero LLM cost), runs the cheap tracker on changed ones, and regenerates fully only on **major** impact. A single-agent design would re-read every repository every night.
3. **Separately measurable.** Verdict accuracy (is **impact** correct?) and documentation quality (grounded, Swedish, well-structured?) are evaluated with different cases and metrics, so regressions are attributable to one agent.
4. **A second deliverable for free.** The tracker's changelog **is** the customer-facing release note (**Ändringshistorik**).
5. **Independent evolution.** The tracker can later gain a Jira or Azure DevOps tool; the analyst can gain a terminology glossary — without touching the other agent's instructions or evaluation set.

1.6 Information flow between the agents

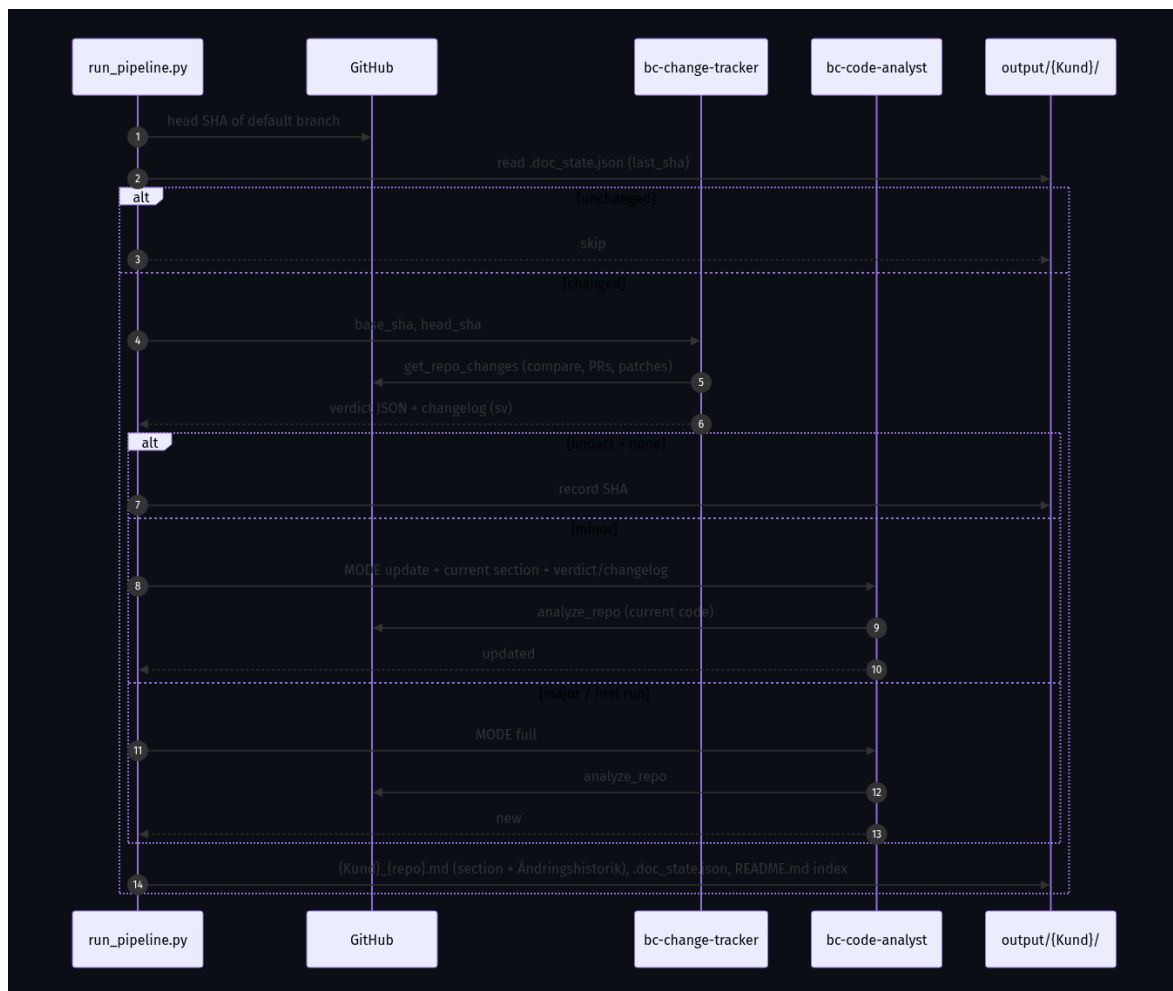


Figure 3 — Sequence per repository: SHA check, change tracker, verdict-driven analyst mode, deterministic assembly.

6. The orchestrator compares the repository's head SHA with the SHA stored in `.doc_state.json`.
7. If it changed, `bc-change-tracker` calls `get_repo_changes(base_sha, head_sha)` and returns a JSON verdict and a Swedish changelog entry.

8. The verdict decides the analyst's mode. In **update** mode the analyst receives the current section and the change summary, calls `analyze_repo` for the current code, and returns the full updated section while keeping unchanged paragraphs — the customer's documentation diff stays small.
9. The orchestrator assembles the document deterministically: metadata header, the agent's `###` section between marker comments, and the `*Ändringshistorik*` with the tracker's entry prepended.
10. The customer index cross-references the BC environment reports from discovery, so one page shows documented version, version in Git and version in each BC environment.

2. Production-readiness plan

2.1 Observability strategy

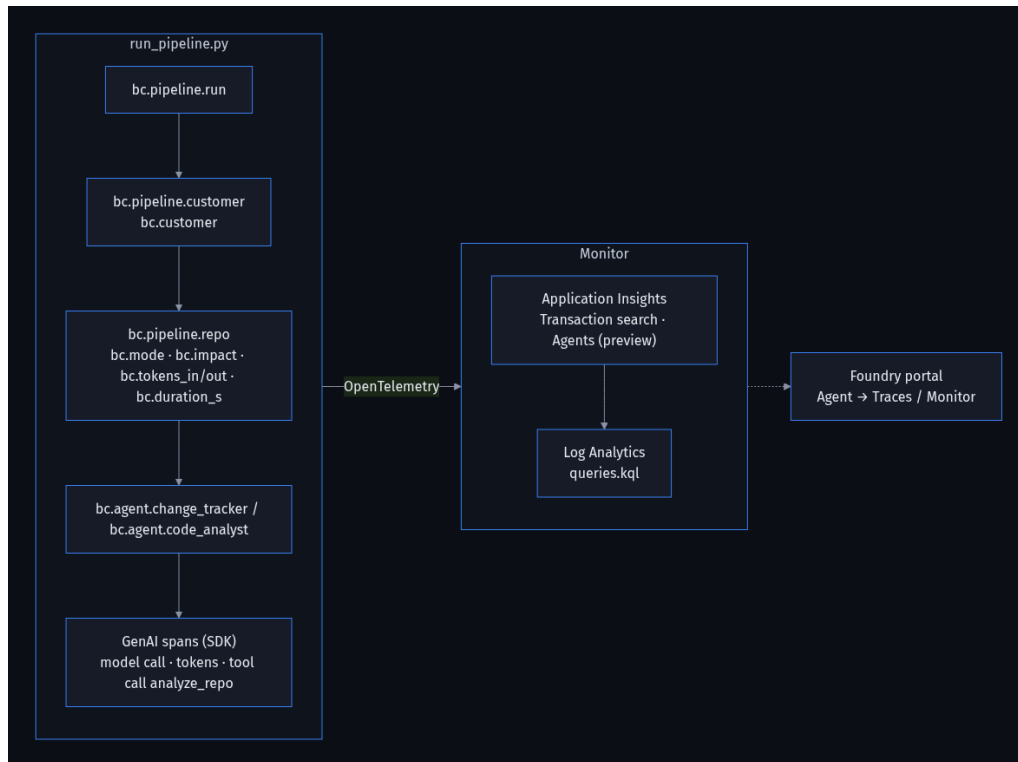


Figure 4 — Span hierarchy exported through OpenTelemetry to Application Insights; the Foundry portal reads the same data.

Traces collected

Span	Attributes	Question it answers
GenAI spans (automatic, <code>AIPProjectInstrumentor</code>)	model, input/output tokens, latency, prompt and completion content, tool call name/arguments/result, content-filter results	Is the model slow or failing? What did the agent actually see and say? Did GitHub time out inside the tool?
<code>bc.pipeline.run</code>	<code>bc.repos</code> , <code>bc.documented</code> , <code>bc.updated</code> , <code>bc.skipped</code> , <code>bc.failed</code> , <code>bc.tokens_in/out</code> , <code>bc.duration_s</code>	Did tonight's run succeed? Alert on <code>bc.failed > 0</code>
<code>bc.pipeline.customer</code>	<code>bc.customer</code> , <code>bc.repos</code> , <code>bc.failed</code>	Per-customer roll-up
<code>bc.pipeline.repo</code>	<code>bc.repo</code> , <code>bc.mode</code> (full/update/skip/no_app), <code>bc.impact</code> , <code>bc.pull_requests</code> , <code>bc.tokens_*</code> , <code>bc.duration_s</code> , <code>bc.section_chars</code>	Cost per repository, mode distribution, slow repositories
<code>bc.agent.change_tracker / bc.agent.code_analyst</code>	<code>bc.repo</code> , <code>bc.mode</code>	Attribute latency and tokens to the right agent

Monitoring metrics

Eight ready-made Kusto queries ship in `monitoring/queries.kql`, all keyed on the `bc.*` custom dimensions:

- Pipeline runs over time with documented / updated / skipped / failed counts
- Token cost per customer (last 30 days) — which customers drive the LLM bill
- Mode distribution per day — how effective the SHA-based change detection is
- Impact verdict distribution per repository — are pull requests mostly minor or major
- Slowest repositories — candidates for a larger source budget or a repository split
- Per-agent GenAI statistics: runs, errors, p50/p95 latency, tokens
- Tool call health: `analyze_repo` and `get_repo_changes` failures and duration
- Failure alert — any run in the last day with `bc.failed > 0`

How the insights are used

Symptom	Where to look	Action
A consultant reports a documented feature that does not exist	Latest <code>bc.pipeline.repo</code> trace → <code>analyze_repo</code> span	If the object is absent from the tool output the model hallucinated → tighten instructions, add an evaluation case. If present but misdescribed the source was truncated (<code>source_omitted_for_budget</code>) → raise <code>MAX_AL_CHARS</code> or split the repository
Rising <code>update/full</code> ratio	Mode distribution query	Developers merge often; consider a PR-triggered run instead of nightly
Tokens per repository trending up	Cost per customer query, <code>bc.section_chars</code>	Repositories are growing; review source budget, consider per-folder documentation
p95 latency spikes	Per-agent GenAI query	Model deployment capacity (TPM) or GitHub API rate limits
<code>bc.failed > 0</code>	Failure alert query → run trace	Exception text is on the span; per-repo isolation means the rest of the run completed

Message-content capture (`OTEL_INSTRUMENTATION_GENAI_CAPTURE_MESSAGE_CONTENT=true`) stores prompts and completions, which include customer AL source. That is what makes debugging possible, so the Application Insights resource is treated as customer-confidential: dedicated resource group, restricted readers, default 90-day retention.

2.2 Evaluation strategy

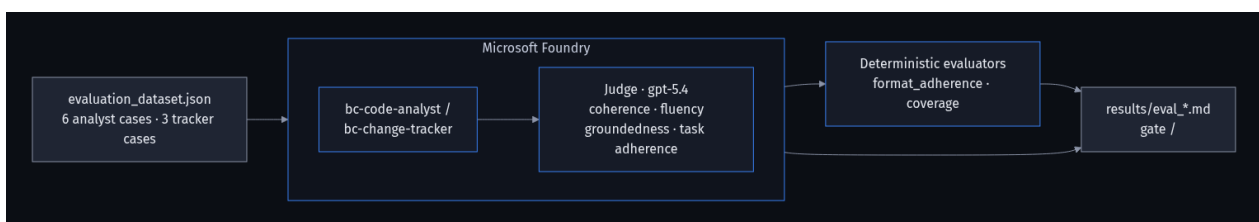


Figure 5 — Evaluation flow: dataset → agents → LLM judge and deterministic evaluators → results and quality gate.

Why evaluate a documentation generator

Monitoring tells you the pipeline ran. Evaluation tells you whether a consultant can trust the output. The failure modes specific to this solution are: **hallucinated features** (a fluent Swedish paragraph about a setup page that does not exist), **format drift** (a model update starts emitting bullet lists, English, or a changelog inside the section), **wrong impact verdicts** (a new codeunit rated `minor`, so the pipeline patches instead of regenerating) and **skipped archetype rules** (install codeunits documented, subscribers not collapsed).

Dataset

`evaluation/evaluation_dataset.json` contains nine cases for a fictional customer, *Nordvik Industri AB*. Each case carries the exact tool output the agent would have received (the analysis JSON or the change JSON), so evaluation runs without GitHub access and is fully reproducible. Each `expected` block holds `must_mention` facts, `required_headings`, `forbidden` phrases, the expected impact, and a Swedish reference answer used as ground truth in the Foundry portal.

Case	Agent	What it tests
CA-01	<code>code_analyst</code>	Logic codeunit + setup table/page + release trigger → its own <code>####</code> section
CA-02	<code>code_analyst</code>	Subscriber-only codeunit plus install codeunit → "Övriga anpassningar", install skipped
CA-03	<code>code_analyst</code>	Commented-out codeunit → "Fungerar ej då logiken ej är implementerad"
CA-04	<code>code_analyst</code>	Processing-only report triggered from a page action
CA-05	<code>code_analyst</code>	API page properties, table extension, setup page, dependency
CA-06	<code>code_analyst</code>	Update mode: integrate a new feature, keep unchanged text, no changelog in the section
CT-01	<code>change_tracker</code>	Minor change via one pull request → <code>minor</code> , no full regeneration
CT-02	<code>change_tracker</code>	New codeunit and new dependency → <code>major</code> , full regeneration
CT-03	<code>change_tracker</code>	Version bump and pipeline files only → <code>none</code>

Criteria

Metric	Type	Why it matters here
Groundedness	LLM judge (context = tool JSON)	The number-one risk: fluent invention of features
Task adherence	LLM judge	Did the agent follow the archetype and format instructions?
Coherence	LLM judge	One feature per section, no contradictions
Fluency	LLM judge	Professional Swedish
Format adherence	Deterministic	###/#### structure, forbidden phrases, JSON verdict block, Swedish markers — runs free in CI
Coverage	Deterministic	Share of required facts (fields, actions, PR numbers) present
Verdict accuracy	Deterministic	impact and requires_full_regeneration match expectations

Tool Call Accuracy is intentionally not used in the portal because local FunctionTools cannot execute there; tool usage is instead verified through the traces (Section 2.1) and by the fact that the deterministic checks cannot pass without the tool's data. The judge uses the same gpt-5.4 deployment through the Foundry account endpoint with Entra ID authentication.

Integration into the development lifecycle

11. **Pre-merge.** GitHub Actions runs the deterministic evaluators (`evaluate.py --offline --no-llm-judge`) on every pull request. Pull requests touching `agents/instructions/` or `common/al_analysis.py` additionally run the full LLM-judged evaluation with a 3.5 threshold gate; the job exits non-zero when any metric mean is below it.
12. **Pre-release.** `agents.py --recreate` publishes a new agent version only after the gate passes. Old versions stay in Foundry for rollback.
13. **In production.** Weekly scheduled portal evaluation on the agents' traces (Foundry → Agent → Monitor → Scheduled evaluations); quality trends next to cost and latency.
14. **Feedback loop.** A consultant flags a document → the repository's analysis becomes a new case → fix → re-evaluate. The dataset grows with the product; the portal can also convert production traces into datasets.

The deterministic gate already runs green on the reference answers:

```
$ python evaluation/evaluate.py --offline --no-llm-judge
[CA-01] Logic codeunit with setup table/page and release trigger    format_adherence=5.0,
coverage=5.0
[CA-02] Subscriber-only codeunit plus install codeunit            format_adherence=5.0,
coverage=5.0
...
[CT-03] No functional change: version bump and pipeline files only format_adherence=5.0,
coverage=5.0
    format_adherence      5.0
    coverage              5.0
Quality gate (>= 3.5): PASSED
```

2.3 Governance and reliability

Consistent outputs

- The **format** is enforced by instruction files versioned in git and verified by deterministic evaluators on every pull request.
- The **content** is bounded by tool output: the analyst cannot document an object that `analyze_repo` did not find.

- The orchestrator assembles the final document itself — header, section between marker comments, change history — so agents never touch metadata or history, and manual notes outside the markers survive regeneration.
- An agent response that does not start with a **###** heading is rejected; an existing document is never overwritten with a malformed one.

Grounding through knowledge sources and tools

Archetype classification (which files become sections, which are collapsed, which are skipped), field and action extraction, event-subscriber parsing, and the Git-versus-Business-Central version validation are all code, not model judgement. The model receives verified facts and phrases them; the instruction files tell it explicitly to say when source was omitted for size rather than guess. Pull request titles and bodies give the tracker the **why** behind a change without inventing intent.

Safe behaviour

- Foundry content filters are active on both agents (default Responsible AI configuration).
- The pipeline only **reads** GitHub and Business Central and only **writes** Markdown into its own repository; there are no write actions against customer systems.
- A human still reviews documents before they are handed to customers; the tool is decision support for consultants, not an autonomous publisher to customers.
- Traces containing customer code live in a dedicated, access-restricted Application Insights resource.

Maintainability and least privilege

Concern	Measure
Structure	Six folders mirror the lifecycle: setup , discovery , agents , monitoring , evaluation , orchestration ; shared code in common
Configuration	Credentials read as environment variable → JSON file fallback; same code runs locally with .env and unattended in GitHub Actions
Prompts	Markdown instruction files, reviewable by consultants, versioned in git, published as immutable agent versions
Idempotency	.doc_state.json makes every run resumable; a second run right after the first costs zero tokens
Failure handling	Per-repository try/except with continue; failures counted, printed, traced; exit code 1 for the scheduler
GitHub access	Dedicated bot account, fine-grained PAT with Contents / Metadata / Pull requests read-only
Business Central access	One app registration in the partner tenant using GDAP delegated administration; customer tenant id per environment; no per-customer secrets
Foundry access	Entra ID (DefaultAzureCredential) everywhere, no API keys; GitHub Actions authenticates with OIDC
Infrastructure	Dedicated resource group created by setup/deploy.sh ; cleanup.sh removes it

3. End-to-end workflow

3.1 Sequence of agent interactions

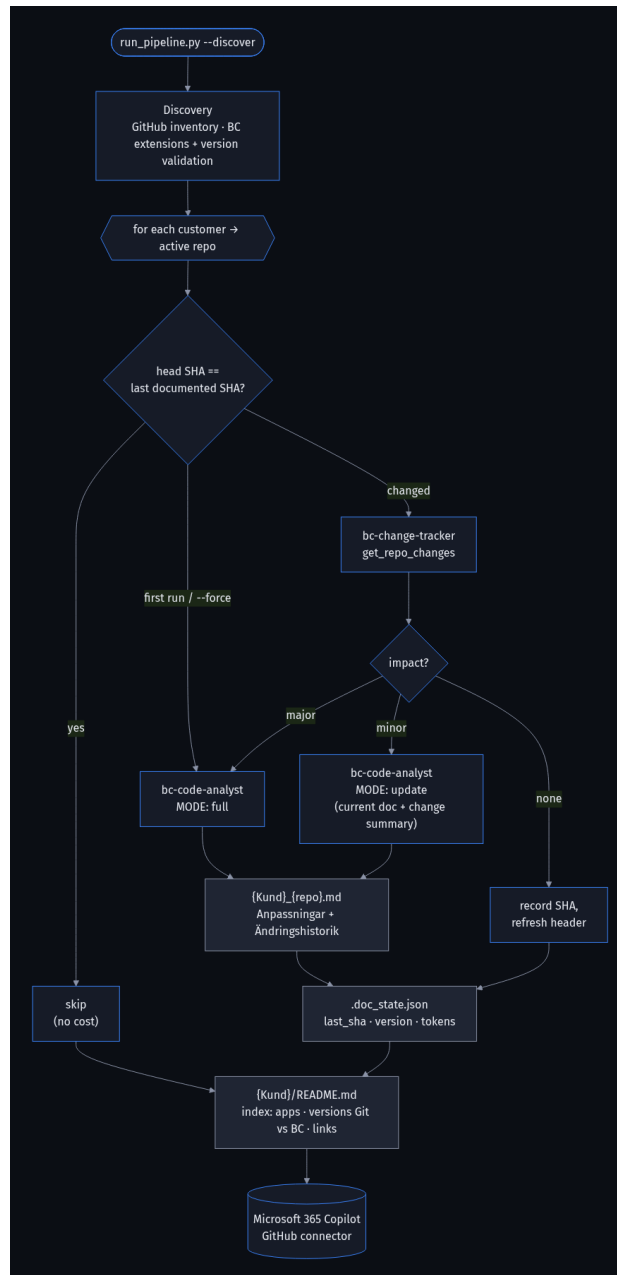


Figure 6 — The production pipeline (`orchestration/run_pipeline.py`): SHA-based change detection routes each repository to skip, update or full regeneration.

15. **Trigger** — nightly GitHub Actions schedule (or manual dispatch; a pull-request-merge webhook is the planned next step).
16. **Discovery** — `discover_repos.py` refreshes the customer inventory with `app.json` metadata and head SHAs; `discover_bc_extensions.py` queries each Business Central environment and writes the **Installerade tillägg** report with version validation.
17. **Change detection** — per repository, head SHA versus `.doc_state.json`.
18. **bc-change-tracker** — `get_repo_changes` → verdict + Swedish changelog.
19. **bc-code-analyst** — `analyze_repo` → full or updated **Anpassningar** section.
20. **Assembly** — one document per app, change history prepended, customer index regenerated with Git-versus-BC versions.

21. **Publish** — commit to the documentation repository → Microsoft 365 Copilot GitHub connector indexes → consultants ask Copilot.
22. **Observe, evaluate, improve** — traces and KPIs in Application Insights; scheduled evaluations; dataset grows from feedback.

3.2 Information passed between steps

From → To	Payload
Discovery → orchestrator	<code>repos_config.json</code> (repositories, app id/name/version, head SHA); BC reports (versions, statuses per environment)
Orchestrator → bc-change-tracker	<code>org</code> , <code>repo</code> , <code>base_sha</code> , <code>head_sha</code>
bc-change-tracker → GitHub (tool)	compare API, merged pull requests, patches for <code>.ai</code> and <code>app.json</code> , changed-object classification
bc-change-tracker → orchestrator	Verdict JSON (<code>impact</code> , <code>affected_features</code> , <code>new_objects</code> , <code>requires_full_regeneration</code> , <code>pull_requests</code> , <code>summary_sv</code>) + changelog Markdown
Orchestrator → bc-code-analyst	Mode: <code>org/repo/branch</code> ; in update mode also the current section and the verdict/changelog
bc-code-analyst → GitHub (tool)	Tree, <code>app.json</code> , all <code>.ai</code> files → structured analysis with archetypes and source under a size budget
bc-code-analyst → orchestrator	### section (Swedish)
Orchestrator → files	<code>{Kund}_{repo}.md</code> , <code>.doc_state.json</code> , customer <code>README.md</code>
Files → Copilot	Markdown indexed by the GitHub connector

3.3 Discovery and version validation

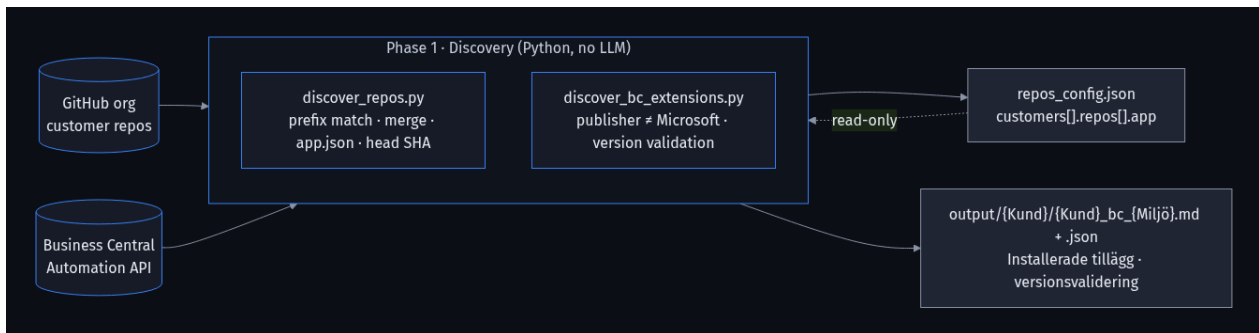


Figure 7 — Discovery is deterministic and isolated: the BC script reads the inventory and writes its own per-environment report.

The Business Central script replaces the proof of concept's manual placeholder ("KRÄVER MANUELL KOMPLETTERING") with a deterministic validation. Own-published apps are matched to repositories by app id first, then by normalised name, and compared on major.minor.build:

Status	Meaning
✅ Matchar	<code>app.json</code> version equals the published version
⚠️ Git är nyare – ej publicerad ändring	Source has moved on; the environment runs an older build
⚠️ BC är nyare – källkod saknar publicerad version	A build is installed that is not in the default branch (hotfix branch, unpushed change)
❓ Inget repo hittades	Own-published app without a matching repository — renamed or unpublished
❓ Ej publicerad i miljön	Repository with <code>app.json</code> that is not installed in this environment

3.4 Final output delivered to the business

Per customer the pipeline maintains a folder that reads like the partner's hand-written implementation document, but never goes stale:

```

output/
├─ Nordvik_Industri_AB/
│  └─ README.md
# index: apps, versions Git vs BC, links,
  
```

```
latest changes
├─ Nordvik_Industri_AB_Nordvik-PTE.md          # Anpassningar + Ändringshistorik for one app
├─ Nordvik_Industri_AB_Nordvik-API-PTE.md
├─ Nordvik_Industri_AB_bc_Production.md         # installed extensions + version validation
├─ Nordvik_Industri_AB_bc_Production.json
└─ .doc_state.json                             # last documented SHA / version / cost per repo
```

Excerpt of a generated app document (fictional customer, from the evaluation dataset):

```
### Nordvik PTE
Kundspecifika anpassningar för Nordvik Industri AB. Appen är beroende av SweBase ...

#### Minimumorderavgift (NVK Calc Minimum Order Fee)
Funktionen lägger automatiskt till en avgiftsrad på försäljningsorder vars artikelsumma understiger ett konfigurerat minimibelopp. Inställningarna görs på sidan NVK Min. Order Fee Setup ... När en order frisläpps (Release Sales Document, OnAfterReleaseSalesDoc) beräknas summan av orderns artikelrader ...

#### Kreditlimitkontroll (NVK Custom Credit Limit Mgt.)
Kundkortet har utökats med fältet NVK Custom Credit Limit ... överstiger summan den egna kreditlimiten stoppas frisläppningen med ett felmeddelande.

#### Övriga anpassningar
- **Externt dokumentnummer från kund** - ... (`SetExternalDocNoFromCustomer`)

## Ändringshistorik
#### 2026-09-10 - version 27.2.0.0 → 27.3.0.0
**Nya funktioner**
- Kreditlimitkontroll: Ny codeunit NVK Custom Credit Limit Mgt. stoppar frisläppning ... (PR #51)
```

Consultants consume this through Copilot chat, for example: `"Vilka anpassningar har Nordvik på inköpsorder och när ändrades de senast?"` — the answer cites the app document and its change history.

3.5 Deployment, monitoring, evaluation and improvement over time

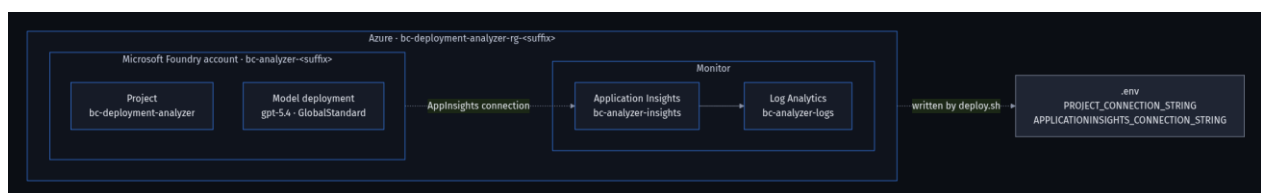


Figure 8 — Infrastructure provisioned by `setup/deploy.sh` in a dedicated resource group.

Aspect	Version 1 (this repository)	Next iteration
Deployment	GitHub Actions nightly (<code>generate-docs.yml</code>), Azure OIDC login, secrets in GitHub; commits <code>output/</code> for the Copilot connector	Pull-request-merge webhook → Azure Function documenting just that repository within minutes
Monitoring	Application Insights traces, <code>bc.*</code> KPIs, KQL queries, failure alert	Azure Monitor workbook per customer; budget alert on tokens
Evaluation	Nine-case dataset, LLM judge + deterministic gate, portal evaluations	Dataset built from production traces; consultant thumbs-up/down feeding new cases
Documentation scope	Section 3 *Anpassningar* + Section 1 extension inventory with version validation	Section 2 *Flödesbeskrivning* from a curated knowledge base (File Search); internal/shared repositories
Agents	Two	Optional reviewer agent scoring each section before publish; Jira/DevOps tool for the tracker
Orchestration	Python orchestrator (function-call loops); Foundry Workflow agent for portal visibility	Foundry Workflow as primary once hosted tools can call GitHub

4. From prototype to production — reflection

The starting point was a working proof of concept: three Python scripts and one plain OpenAI Responses call per repository, with all AL files pasted into the prompt. It produced good Swedish documentation on the first real customer — and it had no way to tell whether the next run would too. The course concepts changed the design in five concrete ways:

Course concept	Prototype	Production architecture
Specialised agents with tools	One prompt, all source pasted in	Two <code>PromptAgentDefinition</code> agents, each with one deterministic <code>FunctionTool</code> ; archetype rules moved from prompt text into code
Knowledge sources and grounding	Model inferred structure from raw AL	Structured analysis JSON; the model cannot cite an object the tool did not find
Tracing and monitoring	Print statements	OpenTelemetry GenAI spans + business KPIs per customer and repository in Application Insights
Evaluation with datasets and metrics	Read the output and judge by eye	Reproducible dataset, four LLM-judge metrics plus deterministic evaluators, CI quality gate
Orchestration into workflows	Regenerate everything every run	SHA-based incremental pipeline; tracker verdict routes to skip / update / full; Foundry Workflow agent for portal visibility

The most valuable shift was treating **what the model is allowed to know** as an engineering artifact. Once the analysis tool decided which codeunits are business logic, which are event subscribers and which are install scaffolding, the agent's job became phrasing rather than judging — and phrasing is something the evaluators can measure. The second shift was accepting that a nightly pipeline is a cost problem: the change tracker exists as much to save tokens as to write release notes.

Status at submission: all phases are implemented and verified offline (module compilation, AL analyzer against the dataset, document renderers, deterministic evaluation gate). Live runs against Foundry, GitHub and Business Central require the partner's credentials and are the first operational step after provisioning with `setup/deploy.sh`.

Appendix A — Repository map

Path	Purpose
<code>README.md</code> , <code>SOLUTION_DESIGN.md</code>	Overview with architecture diagram; the design in Markdown form
<code>setup/deploy.sh</code> , <code>cleanup.sh</code>	Provision / remove the dedicated resource group (Foundry account, project, gpt-5.4, Log Analytics, App Insights)
<code>discovery/discover_repos.py</code>	GitHub inventory with <code>app.json</code> metadata and head SHA
<code>discovery/discover_bc_extensions.py</code>	Business Central extension inventory and version validation report per environment
<code>agents/agents.py</code> , <code>agents/tools.py</code> , <code>agents/instructions/*.md</code>	The two agents, their <code>FunctionTools</code> and instruction files
<code>common/al_analysis.py</code> , <code>common/github_client.py</code> , <code>common/tracing.py</code> , <code>common/config.py</code>	Deterministic AL parser, GitHub client, OpenTelemetry helpers, configuration
<code>monitoring/monitor.py</code> , <code>monitoring/queries.kql</code>	Traced run and Kusto queries
<code>evaluation/evaluate.py</code> , <code>evaluation/evaluation_dataset.json</code> , <code>evaluation/eval_portal.jsonl</code>	Evaluation pipeline, dataset, portal export
<code>orchestration/run_pipeline.py</code> , <code>orchestration/create_workflow_agent.py</code>	Production pipeline; Foundry Workflow registration
<code>.github/workflows/generate-docs.yml</code>	Nightly scheduled run with OIDC login and commit-back
<code>docs/examples/</code>	Rendered sample output for a fictional customer